Chapter 16

UNDOING

16.1 MOTIVATION

To err is human. A practical software system must take into account the fact that its users will make various sorts of mistakes during the use of the system. If no protection against errors is provided, the system is very likely to be abandoned.

To a certain extent, it is possible to avoid errors by introducing shielding mechanisms that protect potentially "dangerous" operations. Unfortunately, shields irritate experienced users while not necessarily helping the naive users.

A better solution is to augment error avoidance techniques with proper error recovery techniques that allow the user to nullify the effects of an erroneous command. The most general form of error recovery is the *undo command* that can remove the effect of any previous operation (except for itself).

Undoing comes in many flavors. Sometimes only limited undoing that can recall some fixed number of previous states is provided. While users will not very often undo more than one or two commands, the full power of undoing only becomes available if an unlimited undo operation that can remove the effect of any number of previous operations is possible. It gives the user the freedom of experimenting with alternatives with the knowledge that a wrong path can always be traced back.

A generalization of the undo command is the *interrupt mechanism* that allows the undoing of an partially executed operation. Suppose the user has started a time-consuming operation, only to notice that it is somehow erroneous. The fact that the operation can be undone does not relieve the user from the pain of first having to wait for the end of the erroneous

operation. In such cases, some sort of an interrupt mechanism that allows the user to interrupt the operation and resume the previous state should be supported.

A properly implemented interrupt mechanism can be used not only by the user, but also by the algorithms of the system itself. Sometimes it is not practical to check for the correctness of the input data to an algorithm until some real computation that alters the state of the system has been done. If the algorithm is unable to continue, it should roll back all changes it has caused. Clearly, an interrupt operation will do the right thing. Going further, an interrupt operation can even be used to protect the system against unexpected internal error conditions, provided that underlying operating system provides support for something equivalent to the signal handlers of the UNIX operating system.

This chapter describes the design and implementation of an unlimited undoing and interrupting mechanism for GWB. In addition to its practical value, the operation will also highlight the real power of the strictly layered architecture of GWB and of the Euler operators. We shall also show how the undo operation can be utilized beyond the mere error recovery.

16.2 OVERVIEW

A prime example of the necessity of undoing is given by real time data base manipulation applications that must be designed so as to preserve the *transaction integrity*. For instance, the sequence of operations required for transfering $100.00 from the bank account of my publisher to my personal account must either be processed completely, or not at all.

The typical solution is based on an *undo log* that records all individual changes to the data base. If the operation must be canceled for some reason, the log can be examined so as to nullify all changes. (For maximal security, the data base is usually modified only after all changes have been written into the log, and the application has issued a special `commit` command.)

Observe that the undo log approach is applicable in this case because the effect of the banking transaction can be adequately represented as a relatively short sequence of simple assignment operations, where some old piece of data is replaced by a new one. Then a sequence of log items of the form

```
<transaction-id, old value, new value>
```

will be sufficient.

For more general computations, the assignment-based undo log approach is usually not applicable because of the prohibitively large bulk

of assignments involved. Clearly, this applies to solid modeling as well: solid modeling algorithms frequently modify quite large amounts of data during a single user-level operation.

To make the undo log approach feasible for applications such as solid modeling, we should replace the simple assignments by some much more powerful basic operations. The operations should be powerful enough to represent the effect of the computation, yet simple enough to have a concise representation in the undo log.

Fortunately for us, Euler operators fill this specification almost exactly. By the theoretical results of Chapter 10, we know that Euler operators are powerful enough to describe all meaningful manipulations of GWB data structures. Moreover, the algorithms of Part Two were implemented (almost) solely by means of the Euler operators. Finally, it is possible to use vertex, face, and solid identifiers to represent Euler operators efficiently in an undo log.

This leads us to the following design for an undo facility for GWB:

1. Design an undo log that can store arbitrary sequences of Euler operators and other basic operations of GWB.

2. Provide all Euler operators with the capability of maintaining the undo log. That is, in addition to manupulating the GWB data structures as usual, each operator is responsible for storing such Euler operators that can undo its effect into the undo log.

3. Of course, most interesting (user level) operations cause more than one Euler operator to be executed. Therefore, we need appropriate facilities for managing modeling transactions consisting of a (potentially long) sequence of basic operations.

The following sections will elaborate on these items.

16.3 AN UNDO LOG DATA STRUCTURE

Most algorithms of GWB of Part Two use the low-level Euler operators which are parameterized in terms of pointers to the GWB data structure. For the undo log, we need an equally powerful set of operators parameterized by means of solid, face, and vertex identifiers. Fortunately, the set of high-level Euler operators as defined in Section 11.5.2 can be used. For reference, the high-level operators and their parameterizations are given in Figure 16.1.

With this parameterization, the undo log data structure becomes very straightforward. As shown in Program 16.1, we can simply use a linked list

```
Id      s, f, f1, f2, v, v1, v2, v3, v4;
float   x, y, z;

Solid   mvsf(s, f, v, x, y, z);
int     kvsf(s);

int     mev(s, f1, f2, v1, v2, v3, v4, x, y, z);
int     kev(s, f, v1, v2);

int     mekr(s, f, v1, v2, v3, v4);
int     kemr(s, f, v1, v2);

int     mef(s, f1, v1, v2, v3, v4, f2);
int     kef(s, f1, v1, v2);

int     kfmrh(s, f1, f2);
int     mfkrh(s, f1, v1, v2, f2);
```

Figure 16.1 The parameterization of high-level Euler operators.

of records consisting of an operation code that identifies the Euler operator, a field for the solid identifier, six other integer fields that are used to store the integer parameters of Euler operators, and four floating-point fields that are used to store the coordinate information. (Of course, a variable-length record could also be used.) The procedure **addeulerop** is intended for adding one Euler operator to the log; its implementation is left to the reader.

Program 16.1 also includes a few global variables which will be used later in this chapter. The variable *OpHead* will point at the head of the undo log (that is, the most recent log record). The variable *OpCount* will store the current number of log records. The global variable *Generatelog* will be used to turn undo logging on and off; if *Generatelog* is zero, no log records will be added to the log.

16.4 UNDOABLE EULER OPERATORS

Let us now look into the task of incorporating the management of the undo log into the low-level Euler operators. Fortunately, in most cases the changes required in the program codes are almost trivial.

As a simple example, the modified code of the **lmev** operator is shown in Program 16.2. The only change from the original version of Program 11.6


```
# define        MVSF    0
# define        KVSF    1
# define        MEV     2
# define        KEV     3
# define        MEF     4
# define        KEF     5
# define        KEMR    6
# define        MEKR    7
# define        KFMRH   8
# define        MFKRH   9

typedef struct eulerop  EulerOp;
struct eulerop
{
        short   opcode;
        Id      solidno;
        Id      ip1, ip2, ip3, ip4, ip5, ip6;
        double  fp1, fp2, fp3, fp4;
        EulerOp *opnext;
};
EulerOp *OpHead;
int     OpCount;
int     Generatelog;

void    addeulerop(opcode, solidno, ip1, ip2, ip3,
                   ip4, ip5, ip6, fp1, fp2, fp3, fp4)
short   opcode;
Id      solidno;
Id      ip1, ip2, ip3, ip4, ip5, ip6;
double  fp1, fp2, fp3, fp4;
{
        ...
}
```

Program 16.1 Undo log data structures and operations.

```
void            lmev(he1, he2, vn, x, y, z)
HalfEdge        *he1, *he2;
Id              vn;
double          x, y, z;
{
        HalfEdge        *he;
        Vertex          *newvertex;
        Edge            *newedge;

        if(Generatelog)
                addeulerop(KEV,
                           he1->wloop->lface->fsolid->solidno,
                           he1->wloop->lface->faceno,
                           he1->vtx->vertexno,
                           vn,
                           0, 0, 0, 0.0, 0.0, 0.0, 0.0);

        newedge = (Edge *) new(EDGE, he1->wloop->lface->fsolid);
        newvertex = (Vertex *) new(VERTEX,
                he1->wloop->lface->fsolid);
        newvertex->vcoord[0] = x;
        newvertex->vcoord[1] = y;
        newvertex->vcoord[2] = z;
        newvertex->vcoord[3] = 1.0;
        newvertex->vertexno = vn;

        he = he1;
        while(he != he2)
        {
                he->vtx = newvertex;
                he = mate(he)->nxt;
        }

        addhe(newedge, he2->vtx, he1, MINUS);
        addhe(newedge, newvertex, he2, PLUS);

        newvertex->vedge = he2->prv;
        he2->vtx->vedge = he2;
}
```

Program 16.2 The undoable lmev operator.

is the guarded call of the procedure **addeulerop**. In this case, the operator kev is stored according to the parameterization of Figure 16.1. (The reader should verify that this operator can indeed undo the effect of lmev.)

Unfortunately, not all cases are as simple. It turns out that making the operator **lkef** undoable poses the most difficult challenge. This is so because the faces joined by the **lkef** operator may have several loops, which will all become attached into a single face after the operation. Therefore, the undo operation must be capable of rearranging the loops back into their original faces.

An undoable implementation of the **lkef** operator is shown as Program 16.3. The solution is to include the "ring move" operator **ringmv** into the scope of undoing, and insert the appropriate operations into the log before inserting the undo record for the **lkef** itself. Now, if the **lkef** operation is undone, the **ringmv** operations will be executed after the **lmef** operation. They will move the loops that used to belong to the face *f2* recreated back to their proper places.

16.5 OTHER UNDOABLE OPERATIONS

Most operations of GWB are carried out by means of Euler operators. Hence, they can be undone by making the Euler operators undoable. However, some algorithms of GWB also manipulate solids directly, without the Euler operators. These include major parts of GWB such as the splitting and the Boolean set operations algorithms. To make them undoable, we must extend the undo facility to cover these manipulations also.

The need for manipulating solids by operations other than Euler operators stems from the fact that splitting and set operations must be capable of dividing a solid into two solids. This task is beyond the scope of the Euler operators used in this text.

Looking back at the final steps of the set operations algorithm (Program 15.15), we can see that the division is carried out by means of the procedure **movefac(f, s)** that moves the face *f* and all faces recursively connected with it to solid *s*. For reference, the relevant part of Program 15.15 is shown as Program 16.4.

Fortunately, to make **movefac** undoable is straightforward. We simply have to store an undo record that requests the connected set of faces of *f* to be moved back to their original solid, which is available as **f->fsolid**. The revised version of **movefac** that implements this is shown in Program 16.5.

Program 16.4 also includes the procedure **revert** that complements its argument solid. Because it is its own inverse, its undoing is trivial and left to the reader. In addition, the set operations algorithm uses the procedure

```
void        lkef(h1, h2)
HalfEdge    *h1, *h2;
{
    Face        *f1, *f2;
    Loop        *l, *l1, *l2;
    HalfEdge    *he;
    Vertex      *v3;

    l1 = h1->wloop;
    f1 = l1->lface;
    l2 = h2->wloop;
    f1 = l2->lface;
    if(Generatelog)
    {
        for(l = f1->floops; l = l->nextl; l != NIL)
        {
            if(l != l2)
                addeulerop(RINGMV, f1->fsolid->solidno,
                    l->ledg->vtx->vertexno,
                    l->ledg->nxt->vtx->vertexno,
                    f1->faceno, f2->faceno,
                    (l == f1->flout ? 1 : 0),
                    0, 0.0, 0.0, 0.0, 0.0);
        }
        addeulerop(MEF, f1->fsolid->solidno, f1->faceno,
            h2->vtx->vertexno, h2->nxt->vtx->vertexno,
            h1->vtx->vertexno, h1->nxt->vtx->vertexno,
            f2->faceno, 0.0, 0.0, 0.0, 0.0);
    }

    /* the rest is the ordinary lkef code */
    ...
}
```

Program 16.3 The undoable lkef operator.


```
    if(op == DIFFERENCE)
        revert(b);
    for(i=0; i<nfaca; i++)
    {
        movefac(sonfa[i+inda], *res);
        movefac(sonfb[i+indb], *res);
    }
    cleanup(*res);
    for(i=0; i<nfaca; i++)
    {
        lkfmrh(sonfa[i+inda], sonfb[i+indb]);
        loopglue(sonfa[i+inda]);
    }
```

Program 16.4 Division of a solid.

updmaxnames that rearranges the face and vertex identifiers of its argument solid; it must also be made undoable.

16.6 TRANSACTION MANAGEMENT

The undo facilities developed so far form only the lowest level of the undo functionality. They must be appended with user-level operations to make undoing available to all GWB algorithms.

First, we need a facility for marking the boundaries of a single modeling transaction that must be treated as a unit of undoing. Typically, the processing caused by one interactive user command would form a single transaction.

The design of Program 16.6 uses a simple cyclically organized array *TransactionLog* for marking transaction boundaries. When a transaction is started (procedure **BeginTransaction**), the current number of Euler operators stored in the undo log is saved into the array, and the indices *TransactionTop*, *TransactionBottom* are updated appropriately. The static variable *TransactionDepth* is used to keep track of nested calls to *BeginTransaction* and *EndTransaction*. In this implementation, no log records are made for the inner transactions.

A complete transaction can be undone in its entirety by means of the procedure **UnDoTransaction**. To actually undo a single Euler operator (or any other undoable basic operation), the procedure calls the procedure **undoop** given in Program 16.7. Of course, the procedure must switch undo logging off.

```
void    movefac(f, s)
Face    *f;
Solid   *s;
{
        if(Generatelog)
                addeulerop(MOVEFAC, s->solidno,
                        f->faceno, f->fsolid->solidno,
                        0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0);
        domovefac(f, s);
}

void    domovefac(f, s)
Face    *f;
Solid   *s;
{
        Loop            *l;
        HalfEdge        *he;
        Face            *f2;

        dellist(FACE, f, f->fsolid);
        addlist(FACE, f, s);
        l = f->floops;
        while(l)
        {
                he = l->ledg;
                do
                {
                        f2 = mate(he)->wloop->lface;
                        if(f2->fsolid != s)
                                movefac(f2, s);
                }
                while((he = he->nxt) != l->ledg);
                l = l->nextl;
        }
}
```

Program 16.5 Undoable division of a solid.


```
int     TransactionLog[MAXUNDO];
int     TransactionTop = 0;
int     TransactionBottom = 0;
int     TransactionDepth = 0;

void    BeginTransaction()
{
        int     i;

        if(TransactionDepth++ > 0) return;
        TransactionLog[TransactionTop] = OpCount;
        TransactionTop++;
        if(TransactionTop == MAXUNDO)
                TransactionTop = 0;
        if(TransactionTop == TransactionBottom)
        {
                TransactionBottom++;
                if(TransactionBottom == MAXUNDO)
                        TransactionBottom = 0;
        }
        Generatelog = 1;
}

void    EndTransaction()
{
        if(TransactionDepth > 0)
                TransactionDepth--;
}

void    UnDoTransaction()
{
        int     nops;

        if(UnDoTop == UnDoBottom) return;
        UnDoTop--;
        if(UnDoTop < 0)
                UnDoTop = MAXUNDO;
        nops = UnDoRecs[UnDoTop].modelops;
        while(OpCount > nops)
                undoop();
        TransactionDepth = 0;
}
```

Program 16.6 Transaction log management.

```
Solid   *undoop()
{
        Solid    *s;
        EulerOp  *op;
        int      Generatelog_save;

        Generatelog_save = Generatelog;
        Generatelog = 0;
        op = OpHead;
        if(op)
        {
                OpHead = op->opnext;
                s = applyop(op);
                free(op);
                OpCount--;
        }
        else    s = NIL;
        Generatelog = Generatelog_save;
        return(s);
}
```

Program 16.7 Undoing single operations.

Finally, the procedure **applyop** of Program 16.8 interprets a single undo log record and executes the appropriate operations. For completeness, we give the whole procedure, although we have not discussed all operations covered by the procedure.

16.7 APPLICATIONS OF UNDOING

The undoing facility developed in the preceding sections is useful for supporting an interactive undo command invoked by a user. Besides this, there are many other ways of using the undoing machinery.

16.7.1 Error Rollback

A straightforward application of the undoing functionality is the rollback of modifications after a runtime error has been encountered within an algorithm.

Suppose, for instance, that the set operations algorithm notes an irrevocable error after considerable changes to the objects have already been made. In this case, the algorithm can easily use the undoing for canceling all modifications it has made to the argument solids up to the point of the error. To avoid interference with the user-level transaction management, the algorithm can itself mark the initial state by including the following C statements before any other processing:

```
int Generatelog_save, OpCount_save;

Generatelog_save = Generatelog;
Generatelog = 1;
OpCount_save = OpCount;
```

By this, the algorithm can use undo facilities whether undoing is enabled or not.

If an error is encountered, changes performed so far can be rolled back as follows:

```
while(OpCount > Opcount_save)
     undoop();
Generatelog = Generatelog_save;
```

Of course, this technique is not restricted to error recovery, but may also be used for more general purposes. The next section will give an extended example of this.

```
Solid        *applyop(op)
EulerOp      *op;
{
    Solid    *s, *sh;

    if(op->solidno > -1 && op != MVFS)
        s = ssolid(op->solidno);
    s2 = s;
    switch(op->opcode)
    {
        case RINGMV:
            ringmv(s, op->ip1, op->ip2, op->ip3, op->ip4, op->ip5);
            break;
        case REVERT:
            revert(s);
            break;
        case MODIFYNAMES:
            modifynames(s, op->ip1, op->ip2);
            break;
        case MOVEFAC:
            movefac(fface(s, op->ip1), ssolid(op->ip2));
            redo(s);
            redo(ssolid(op->ip2));
            break;
        case MVFS:
            s2 = mvfs(op->solidno, op->ip1, op->ip2,
                      op->fp1, op->fp2, op->fp3);
            break;
        case KVSF:
            kvfs(op->solidno);
            s2 = NIL;
            break;

        /* other operators */
        ...
    }
    return(s2);
}
```

Program 16.8 Applying undo log records.

16.7.2 File Operations of Solids

As described so far, GWB does not include procedures for storing an instance of the half-edge data structure onto secondary storage (disk), or for reading a previously stored data structure back. In this section we shall show how the undo machinery can be used to fill this gap.

Approaches to File Representations of Solids

In principle, the designer of filing operations for a boundary modeler such as GWB has several alternative strategies to choose from. First, it would be possible to store the construction sequence used to generate the solid in the first place. While this approach usually leads to rather concise data base formats, the computational cost of re-evaluating the model can be large if procedures such as the Boolean set operations are used in the construction. Moreover, in order to make data transfer between several programs possible, all programs would have to be capable of evaluating all possible construction operations. Still worse, as the operations stored are peculiar to the particular modeling system, the representation cannot support transfer of information to another modeler.

Clearly, a file format which is independent of the construction history of the solid avoids these problems. One alternative is to directly convert the run-time data structure of the solid into a suitable "pointer-free" external format. To regenerate the object, another conversion back to central storage format is needed. Alternatively, relocatable data structures can be used to represent the solid model in the identical fashion both in central storage and in secondary storage. In this case, transfer of data between central and secondary storage could be handled by a storage management utility. The drawback of a direct conversion is the inherent complexity of modifying a "pointer-rich" data structure such as the half-edge data structure into a file format. Moreover, the external data structure easily becomes voluminous.

The third alternative is the indirect conversion that tries to avoid the problems of direct conversion by mapping the internal data structures to a format particularly useful for file storage. The problem that remains in this approach is the selection of a format that exhibits a good balance between the complexity of generation and retrieval and the size of the external representation.

Solid Inversion

The approach to be followed here can be viewed as a combination of the construction history approach and the indirect conversion approach.

To grasp the basic idea, it is useful to recall the analogy of Chapter 8 between computer languages and Euler operators. This analogy considers the Euler operators an "assembler" on top of which all higher-level operations "execute." In this view, all models of GWB are the result of executing a sequence of Euler operators.

According to the construction history approach, the model could be stored by storing the sequence of operators used to generate the model. The problem that remains is that the sequence may be much longer than necessary. If, say, one of the high-level machines that execute on top of Euler operators is the set operations algorithm, large numbers of faces, edges, and vertices are created and thrown away.

Ideally, we need a "disassembler" that can examine an instance of the half-edge data structure and create a "short" sequence of Euler operators capable of creating it. Then, we could use the "short" as a representation of the object in secondary storage.

At this point, the undo mechanism comes into play. Imagine that we have a procedure capable of erasing a solid by means of the Euler operators. Because the procedure uses Euler operators only, it is undoable—that is, after the procedure has finished, the undo log will contain a sequence of operators that will recreate the solid if executed. Hence, we have indeed found a "disassembly" of the original solid!

We can now immediately give an outline of the file generation algorithm in Program 16.9. The two procedures `remedg` and `remfac` constitute the solid removal algorithm; they are described in the next section. After that, the usual undo loop is applied, except that all operators are first written into a file by means of the procedure `writeop`. This procedure is not elaborated on here.

Solid Removal Algorithm

The problem of removing an instance of the half-edge data structure by means of Euler operators was already encountered in Chapter 9 in the context of proving the completeness of plane model manipulation operations. In fact, the algorithm to be presented can be viewed as a mechanization of the proof of Theorem 9.2, page 141.

The removal is best implemented in two phases. First, all edges of the solid are removed (procedure `remedg` in Program 16.9). After that, the solid will have been reduced to a rather peculiar skeletal object with one lone vertex for each connected component of edges in the original solid. This object is removed in the second phase (procedure `remfac`).

The task of the procedure `remedg` is well represented in Figure 9.2, page 142. Analogously to the proof of Theorem 9.2, the procedure can

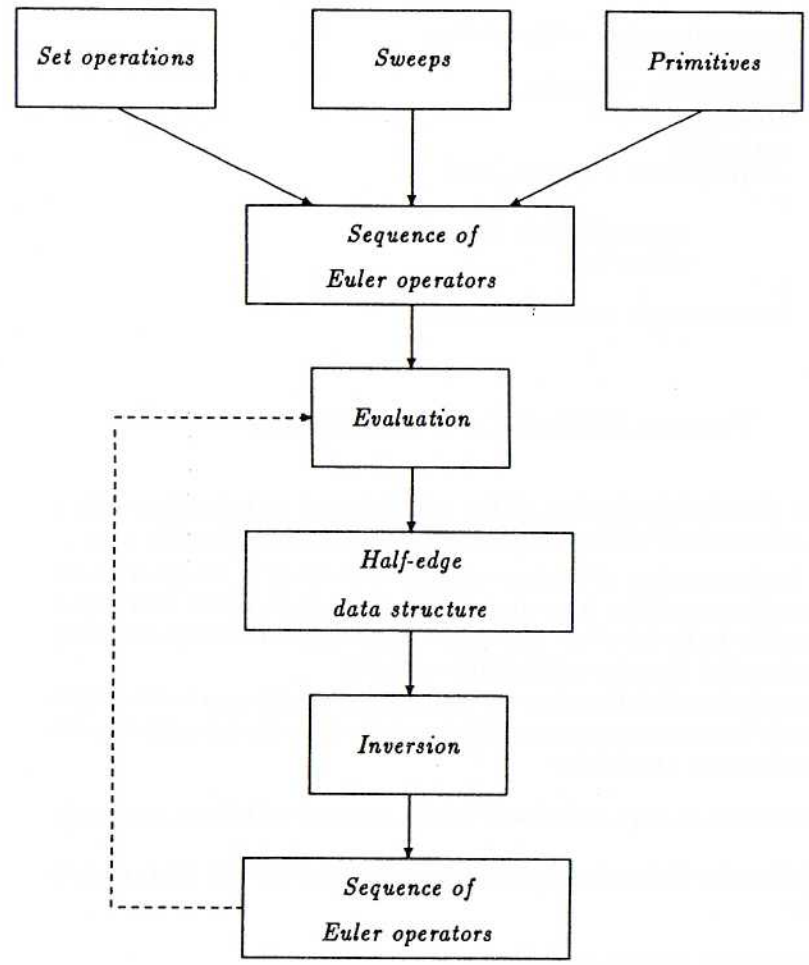

Figure 16.2 The inversion approach.

```
void    invert(s, file)
Solid   *s;
FILE    *file;
{
        int     Generatelog_save, OpCount_save;

        Generatelog_save = Generatelog;
        Generatelog = 1;
        OpCount_save = OpCount;
        remedg(s);
        remfac(s);
        while(OpCount > Opcount_save)
        {
                writeop(OpHead, file);
                undoop();
        }
        Generatelog = Generatelog_save;
}
```

Program 16.9 Outline of the inversion algorithm.

work by considering all edges of the solid in turn, and deciding with a simple case analysis which Euler operator to use for removing the edge.

An implementation of the case analysis is outlined in Program 16.10. The procedure **nextedge** is assumed to supply a pointer to the next edge e of the solid s to be removed. The algorithm examines the loops and faces of e, and applies the appropriate Euler operator.

The order in which the edges are removed has significance to the length of the resulting operator sequence. Fortunately, the rules for achieving the shortest sequence are simple:

1. If these is an edge such that it can be removed with `lkev`, remove it.

2. Otherwise, if there is an edge that can be removed with `lkef`, remove it.

3. Otherwise, remove any edge.

These rules could be implemented in the procedure **nextedge**.

After the edges have been removed, each shell (connected set of faces) of the original solid has shrunk to a single face, consisting of a one-vertex loop for each original connected set of edges of that shell. The face removal phase


```
void    remedg(s)
Solid   *s;
{
        Edge            *e;
        Loop            *l1, *l2;

        while(e = nextedge(s))
        {
                l1 = e->he1->hloop;
                l2 = e->he2->hloop;
                if(l1 == l2)
                {
                        if(e->he1->nxt == e->he2)
                                lkev(e->he2, e->he1);
                        else if(e->he2->nxt == e->he1)
                                lkev(e->he1, e->he2);
                        else    lkemr(e->he1, e->he2);
                }
                else if(l1->lface != l2->lface)
                        lkef(e->he1, e->he2);
                else    lmfkrh(l1->lface,
                            (l1 == l1->lface->flout) ? l1 : l2);
        }
}
```

Program 16.10 Removal of edges.

will reduce each shell to a one-vertex "skeletal" shell, and will combine the shells with each other until a single one-vertex shell remains.

The procedure **remfac** that implements this manipulation is shown as Program 16.10. The algorithm first joins the "inner" loops of all remaining faces with their corresponding "outer" loops by means of **lmekr**'s, and deletes the ring vertex with **kev**'s. In the last **while** loop, all faces except one are removed by first reducing them to rings with **kfmrh**'s, and applying the same operators as above.

Reading an Inversion

The inversion algorithm creates a file consisting of a sequence of Euler operators as written by the procedure **writeop**. In practice, **writeop** may well use variable length records to represent Euler operators in secondary storage—say, coordinate data is written only if *opcode* = MEV or MVFS.

Whatever encoding tricks are used in the file, a procedure **readop** must exist such that it is capable of reconstructing Euler operators into an undo log record. Based on **readop**, it is easy to write a procedure that reads an inversion file and returns a pointer to the solid read. It is outlined in Program 16.11.

Ordinarily inversions would be written and read in binary format for greater space efficiency. However, if inversions must be transmitted from one computer to another, binary files can be replaced by ASCII files just by modifying the procedures **writeop** and **readop**.

Notes

The inversion algorithm can be viewed as a constructive proof of Theorem 9.2: every correct instance of the half-edge data structure can be created with a sequence of Euler operators. In fact, because the algorithm works without the operators KVFS, KEV, or KEF, it shows the slightly stronger result that none of these "negative" operators are needed in such a sequence.

Due to this fact, the performance of the algorithm can easily be analyzed. Because no "negative" operators appear, the operator counts of Equations 9.5 will hold for all but KEMR and MEKR. That is, in the absence of rings and holes, a solid whose with v vertices, e edges, and f faces that form a connected surface is represented with exactly with the following numbers of operators:

$v-1$	MEV's
$f-1$	MEF's
1	MVFS


```
void    remfac(s)
Solid   *s;
{
        Face            *f1, *f2;
        Loop            *out, *ring;

        f1 = s->sfaces;
        while(f1)
        {
                out = f1->flout;
                ring = f1->floops;
                while(f1->floops->nextl)
                {
                        if(ring != out)
                        {
                                lmekr(out->lhedg, ring->lhedg);
                                lkev(out->lhedg, ring->lhedg);
                                ring = f1->floops;
                        }
                        else    ring = ring->nextl;
                }
                f1 = f1->nextf;
        }
        f1 = s->sfaces;
        while(f2 = f1->nextf)
        {
                lkfmrh(f1, f2);
                lmekr(f1->floops->lhedg, f1->floops->nextl->lhedg);
                lkev(f1->floops->lhedg, f1->floops->nextl->lhedg);
        }
}
```

Program 16.11 Removal of faces.

```
Solid    *iread(filename)
char     *filename;
{
        FILE            *file;
        Solid           *s;
        EulerOp         op;

        file = fopen(filename, "r");
        while(readop(file, &op))
                s = applyop(s, op);
        fclose(file);
        return(s);
}
```

Program 16.12 Reading an inversion.

Hence, a total of

$$v + f - 1$$

operators would be required in this simple case.

In the general case, rings and holes complicate the counting, because both KEMR, KFMRH and their inverses MEKR, MFKRH are needed in the general case. Observe, however, that MEKR's will only needed in objects that have holes, and that at most one MEKR for each hole is needed. Hence, the number m of MEKR's required in an inversion must be less than h, the number of holes.

Let v, e, f, s, h, and r denote the number of vertices, edges, faces, shells, holes, and rings in the solid, and let m be the number of MEKR'S required, $0 \leq m \leq h$. Then the number of operators required in an inversion can be counted follows:

1. By examination of the algorithm, it is clear that 1 MVFS, $s - 1$ MFKRH's, and h KFMRH's are always generated. These operators generate s shells and h holes; hence shells and holes are all accounted for.

2. The operators above create exactly one vertex. Because all vertices are created, and no vertices created are ever removed, the inversion must include $v - 1$ MEV's.

3. By a similar argument for faces, the inversion must contain $f - s + h$ MEF's.

4. Assuming that there are m MEKR'S, we can use the same argument for rings as well. Hence, the operators already accounted for create $s - h + 1 - m$ rings; since r rings must be created, a total of $r - s + h - 1 + m$ KEMR's are needed.

The account above includes all operators that may appear in an inversion. As a check for the derivation, the reader can verify that these operators create

$$v + f - 2(s - h) - r$$

edges, in accordance with the Euler-Poincaré formula 9.2, page 154. Summing the account, we arrive at the following result:

Theorem 16.1 *A half-edge data structure with v, e, f, s, h, and r vertices, edges, faces, shells, holes, and rings can be created with the following counts of Euler operators:*

$v - 1$	MEV's
$f - s + h$	MEF's
1	MVFS
m	MEKR's
$r - s + h - 1 + m$	KEMR's
h	KFMRH's
$s - 1$	MFKRH's

or a total of

$$v + f + s + h + r - 2 + 2m \tag{16.1}$$

operators where $0 \leq m \leq h$. In particular, no operators KEV, KEF, *and* MVFS *are needed.*

Hence the length of the operator sequence generated by the inversion algorithm grows linearly with the size of the solid.

In practice, some storage is needed for the auxiliary operator RINGMV. Their number can be minimized by removing empty loops immediately as they are created, instead of carrying them along to the face removal phase. With this simple optimization, the storage needed for the operator sequence is in typical cases roughly 30 percent of that of the complete half-edge data structure.

BIBLIOGRAPHIC NOTES

This chapter is mainly based on the papers [74] and [76]. Hence, odd as it may seem, inversion predates undoing.

Instead of a single undo log, it would be perfectly possible to use a per-solid log. This would make it possible to undo operations on a per-solid basis. Another example of a more involved undo mechanism that also supports a "redo" operation is described in [26]. Here the undo logs are organized according to a CSG tree.

PROBLEMS

16.1. Write a procedure **printproperties(s)** that calculates the number of components and the genus of each component of the argument solid, *s*.

Hint: Modify the procedure **remedg** so that it will not change the genus of the solid. After the modified algorithm has finished, the required outputs can be directly deduced from the data structure.

16.2. Write a procedure **scopy(s)** that creates a copy of its argument solid *s*.

Hint: Modify the procedure **invert**.

16.3. How would do undo the copying of a solid performed by the procedure **scopy(s)** of Problem 16.2?

16.4. Consider alternative approaches of undoing the operation **strans(s, m)** that transforms solid *s* according to the 4-by-4 transformation matrix *m*.

16.5. Describe how the undoing facilities described in this chapter could be used to support the interrupt operation that allows the user to stop a computation and restore the state preceding it. (You may assume that you can specify a procedure that will be asynchronously called if the user hits the interrupt key.)